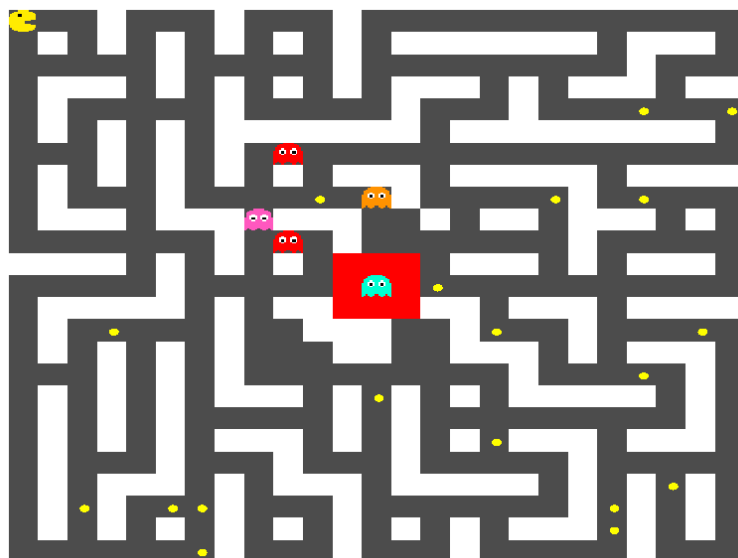


Université Marie & Louis Pasteur
UFR Sciences et Techniques

Projet L3 : Pacman en réseau

Rapport Agile

Licence CMI Informatique - 3ème année
2025-2026



Thomas COUTANT
Benoît LITAMPHA
Auriane PETER

Encadrant : Julien BERNARD

Table des matières

Introduction	2
1 Organisation du projet et approche Agile	2
2 Les Sprints	3
2.1 Sprint 1 : Mise en place de l'architecture complète	3
2.2 Sprint 2 : Développement du client et intégration des bots	3
2.3 Sprint 3 : Stabilisation, amélioration visuelle et débogage intensif	4
3 Analyse critique de l'approche Agile adoptée	4
4 Bilan et retour d'expérience	5
Conclusion	6

Introduction

Le projet de Licence 3 Informatique avait pour objectif la conception et le développement d'un jeu multijoueur en réseau inspiré du jeu d'arcade Pac-Man. Ce projet mobilise de nombreuses compétences techniques tel que l'architecture client-serveur, synchronisation réseau, génération procédurale de contenu, intelligence artificielle et conception d'interface graphique.

Le principal défi technique résidait dans la mise en place d'un client-serveur autoritaire capable de gérer plusieurs connexions simultanées tout en garantissant la cohérence du jeu.

Le projet s'est déroulé en deux grandes phases. Une première phase, de mi-octobre à décembre, a permis de concevoir un prototype minimal client-serveur servant de base technique. Une seconde phase plus intensive, organisée en trois sprints d'une semaine durant le mois de janvier, a permis de transformer ce prototype en une version fonctionnelle et stable du jeu.

Ce rapport présente l'organisation Agile adoptée, le déroulement des différents sprints, les difficultés rencontrées ainsi que les adaptations mises en place.

1 Organisation du projet et approche Agile

Le projet a été en suivant une organisation inspirée des méthodes Agile, avec une progression de type incrémentale afin de valider régulièrement les choix techniques et d'adapter le développement aux difficultés rencontrées. Plutôt que de se baser sur un cahier des charges figé, bien qu'il soit défini, l'équipe a avancé par itérations successives, chaque sprint visant un objectif fonctionnel précis et donnant lieu à une version exécutable, même partielle. Cette approche a réduit les risques liés à l'architecture réseau et évité l'accumulation d'erreurs détectées trop tardivement, tout en s'inspirant du cadre Scrum qui est adapté au contexte universitaire et qui repose sur des cycles courts et une communication continue.

L'équipe était composée de trois étudiants, les tâches ont été réparties comme il suit :

- Thomas s'est chargé principalement du développement serveur, incluant la gestion des connexions, la logique centrale, la génération procédurale et les intelligences artificielles.
- Auriane a développé le client, gérant l'interface graphique, l'affichage et les interactions utilisateur.
- Benoît est intervenu à la fois sur client et serveur, avec un focus sur la communication réseau et la cohérence des échanges.

Les décisions techniques majeures étaient prises de manière collaborative pour assurer la cohérence globale du projet.

Lors de nos sprints, des réunions hebdomadaires avec le tuteur du projet ont permis de présenter l'avancement, valider les choix et ajuster les priorités.

La communication a été continue, principalement via Discord, pour coordonner rapidement les développements et résoudre les problèmes techniques. La gestion du code a été assurée par le gestionnaire de versionnement Git, pour faciliter le travail en parallèle, le suivi de l'évolution et le maintien d'un historique clair des modifications. Chaque sprint suivait un cycle défini : définition des objectifs en début de semaine,

développement parallèle des composantes, communication régulière pour synchroniser client et serveur, point intermédiaire avec le tuteur, puis stabilisation et validation en fin de sprint. Cette organisation a permis d'identifier rapidement les blocages techniques, notamment lors de la refonte du prototype serveur pour le lobby ou de la mise en place du gestionnaire de scènes côté client.

2 Les Sprints

2.1 Sprint 1 : Mise en place de l'architecture complète

Le premier sprint a constitué une étape pour poser une nouvelle base. il visait à transformer le prototype minimal en une vraie architecture client-serveur complète et structurée. L'objectif était de permettre la gestion de connexions multiples, l'intégration d'un système de lobby, la création et la gestion de rooms, l'instanciation de parties indépendantes, l'exécution d'une boucle de jeu serveur ainsi que la génération procédurale du plateau.

Le prototype initial se limitait à une connexion simple avec échanges de messages basiques, sans gestion multi-parties ni organisation modulaire. La principale difficulté a donc été de refondre cette base sans introduire de régressions. Le serveur a été restructuré autour de modules distincts : gestion des connexions, lobby, rooms, instances de jeu et boucle principale. Cette séparation des responsabilités a clarifié la logique métier et limité le couplage interne.

L'introduction du lobby a nécessité une réorganisation des états serveur (connecté, lobby, en partie) et l'isolation complète des instances de jeu, chaque room disposant de sa propre boucle et de son propre état. La boucle serveur, le cœur de l'architecture autoritaire, centralisait les décisions logiques et la synchronisation. Malgré des difficultés liées aux transitions d'états et à la synchronisation lors de créations simultanées de rooms, les itérations successives ont permis de stabiliser l'ensemble. À la fin du sprint, l'architecture était modulaire, le lobby fonctionnel et les bases techniques solidement établies pour la suite du développement.

2.2 Sprint 2 : Développement du client et intégration des bots

Le second sprint avait pour objectif de rendre le système réellement jouable. Après la stabilisation de l'architecture serveur, cette phase s'est concentrée sur la conception d'un client structuré et sur l'intégration de bots contrôlés par intelligence artificielle. Les priorités étaient la mise en place d'une architecture client claire, l'implémentation d'un gestionnaire de scènes (menu, lobby, room, partie), la synchronisation fiable avec l'état autoritaire du serveur et le développement de comportements autonomes pour les bots. Cette étape renforçait l'interdépendance entre client et serveur, rendant la communication réseau centrale.

Le client a été organisé en trois couches distinctes : réseau, logique locale et interface graphique, afin de limiter le couplage et faciliter la maintenance. Le gestionnaire de scènes a structuré le cycle de vie de l'application en garantissant qu'une seule scène soit active à la fois, améliorant la lisibilité et la robustesse du code.

La synchronisation a constitué un enjeu majeur : le client envoyait uniquement les intentions du joueur et affichait les états validés par le serveur. Les ajustements réguliers du format des messages ont permis d'assurer une compatibilité stable. Les bots, intégrés

côté serveur, étaient mis à jour dans la boucle principale, assurant une cohérence totale entre entités humaines et artificielles.

Malgré des difficultés liées au gestionnaire de scènes et aux adaptations continues du protocole réseau, les itérations courtes ont permis de stabiliser l'ensemble. À la fin du sprint, le projet disposait d'un client fonctionnel, de bots intégrés et d'un système jouable de bout en bout, marquant le passage à une expérience interactive complète.

2.3 Sprint 3 : Stabilisation, amélioration visuelle et débogage intensif

Le troisième et dernier sprint a été consacré à la stabilisation du système et à l'amélioration de l'expérience utilisateur. Contrairement aux phases précédentes, centrées sur l'architecture et l'intégration des fonctionnalités, cette étape visait la qualité, la robustesse et la finalisation du projet : correction des incohérences client-serveur, fluidité de l'interface, ergonomie optimisée et stabilisation de la boucle de jeu et des transitions d'états.

Côté serveur, des anomalies apparaissaient lors de tests multi-joueurs, avec des incohérences entre lobby et partie, des erreurs lors des déconnexions et des désynchronisations d'entités. La refactorisation des variables globales et des transitions d'états a amélioré la lisibilité, réduit le couplage et éliminé les comportements indéterminés.

Côté client, l'accent a été mis sur la fluidité et la cohérence : optimisation des transitions entre scènes, amélioration de la réactivité visuelle et harmonisation graphique. Le gestionnaire de scènes a été affiné pour éviter les rechargements inutiles et renforcer la robustesse du traitement des messages réseau, notamment dans les cas limites comme les reconnexions rapides, changements de room ou parties interrompues.

La communication réseau a été simplifiée pour réduire la complexité côté client et limiter les erreurs de parsing, aboutissant à un protocole plus stable et cohérent. L'interface graphique a été améliorée avec un affichage du plateau optimisé, des menus réorganisés et une navigation dans le lobby plus intuitive, renforçant la qualité perçue du projet.

L'approche Agile a permis de consolider progressivement les corrections plutôt que de les accumuler en fin de projet. Les validations hebdomadaires avec le tuteur ont confirmé la stabilité générale et orienté les derniers ajustements. À l'issue de ce sprint, le projet disposait d'une architecture client-serveur stable, d'un lobby fonctionnel, d'une gestion fiable des rooms, d'une boucle de jeu robuste, d'une interface finalisée et d'une expérience de jeu complète, prête à être présentée et évaluée.

3 Analyse critique de l'approche Agile adoptée

L'organisation inspirée des méthodes Agile s'est révélée particulièrement pertinente au regard de la nature du projet. Le développement d'un jeu multijoueur en réseau implique de fortes interdépendances entre le client, le serveur, le protocole de communication, la logique de jeu et l'interface graphique, chacun de ces éléments devant évoluer de manière coordonnée. Le découpage en sprints courts a permis de valider progressivement les choix d'architecture, de détecter rapidement les erreurs structurelles, de limiter l'accumulation de dette technique et d'assurer une montée en complexité maîtrisée. Chaque fin de sprint correspondait à une version fonctionnelle, même partielle, offrant une visibilité concrète sur l'avancement du projet.

Concrètement, cette approche a d'abord permis de sécuriser l'architecture serveur avant toute complexification, évitant ainsi de développer un client élaboré sur des bases instables. Lors de la refonte du prototype pour intégrer le lobby, les ajustements structurels nécessaires ont pu être réalisés sans remettre en cause l'ensemble du système grâce au travail en itérations courtes. Les problèmes de synchronisation client-serveur ont également été résolus progressivement, en adaptant le format des messages et la gestion des états au fil des tests. La communication continue a joué un rôle déterminant pour maintenir la cohérence entre les composants et résoudre rapidement les incompatibilités liées au protocole.

Toutefois, l'organisation adoptée présentait certaines limites. Si GitHub a été utilisé pour le versionnement du code, l'absence d'un outil formel de gestion de tâches, tel qu'un tableau Kanban structuré, a réduit la visibilité sur la charge de travail et la priorisation des objectifs. Par ailleurs, certaines tâches ont été sous-estimées, notamment la refonte du serveur pour intégrer le lobby et la mise en place du gestionnaire de scènes côté client. Cette difficulté d'estimation s'explique en grande partie par la forte interdépendance des composants techniques, caractéristique des architectures client-serveur complexes.

Plusieurs enseignements peuvent être tirés de cette expérience. La mise en place précoce d'un prototype permet de réduire significativement les incertitudes techniques, tandis qu'une architecture serveur modulaire facilite les évolutions ultérieures du système. La séparation claire entre logique métier et interface graphique apparaît également essentielle pour garantir la maintenabilité du code. Enfin, l'organisation en sprints courts favorise une réactivité accrue face aux difficultés rencontrées. De manière générale, cette expérience confirme la pertinence de l'approche Agile pour des projets impliquant de fortes interactions entre composants logiciels, car elle permet de concilier structuration et flexibilité dans un contexte de développement complexe.

4 Bilan et retour d'expérience

Le projet a constitué une mise en pratique approfondie de compétences techniques dépassant largement le cadre d'un simple développement applicatif. La conception d'une architecture client-serveur autoritaire a représenté une étape structurante, permettant d'appréhender concrètement les enjeux de synchronisation réseau, de cohérence d'état et de répartition des responsabilités entre client et serveur. La mise en place du système de lobby et de rooms a introduit des problématiques d'organisation interne, notamment en matière de gestion des états et d'isolation des instances de jeu. Côté client, l'implémentation d'un gestionnaire de scènes a souligné l'importance d'une architecture logicielle claire afin de limiter le couplage entre interface graphique et logique métier. L'intégration des bots a, quant à elle, offert l'opportunité d'explorer des mécanismes d'intelligence artificielle simples mais efficaces, intégrés dans une boucle serveur centralisée.

Au-delà des aspects techniques, le projet a également constitué une application concrète d'une organisation Agile. La structuration en sprints a permis une montée en complexité progressive, une validation régulière des choix techniques et une adaptation rapide face aux difficultés rencontrées. La distinction entre phase de prototypage et phase de développement intensif s'est révélée particulièrement pertinente : le prototype initial a servi de terrain d'expérimentation, réduisant significativement les incertitudes lors des phases ultérieures. L'expérience a également mis en évidence l'importance d'une communication continue dans un contexte de forte interdépendance technique,

la coordination entre client et serveur nécessitant des ajustements constants.

Cette réalisation a favorisé le développement de compétences clés telles que la conception d'architectures modulaires, la gestion de la communication réseau, l'implémentation d'une boucle de jeu centralisée, la structuration d'interfaces graphiques dynamiques et le travail collaboratif sur un dépôt partagé. Elle a également renforcé la capacité d'analyse et de résolution de problèmes complexes, notamment lors des phases de débogage intensif.

Avec un regard critique, plusieurs axes d'amélioration peuvent être envisagés pour un projet similaire : la mise en place d'un outil formel de gestion de tâches, une définition plus précise des critères de validation de chaque sprint, une formalisation plus rigoureuse du protocole réseau dès les premières étapes et l'intégration de tests automatisés afin de sécuriser les refactorisations. Ces évolutions contribueraient à renforcer la robustesse organisationnelle et technique de l'ensemble.

Enfin, ce projet représente une synthèse des compétences acquises au cours de la formation et illustre la complexité réelle d'un développement logiciel impliquant architecture distribuée, gestion d'état et interaction utilisateur. L'approche Agile adoptée a permis de transformer une idée initiale en un système cohérent et fonctionnel malgré les contraintes temporelles et techniques, constituant ainsi une base solide pour aborder des projets de plus grande envergure avec rigueur méthodologique et maîtrise technique.

Conclusion

Le développement du jeu Pac-Man multijoueur en réseau a constitué une expérience complète d'ingénierie logicielle, mobilisant à la fois des compétences techniques avancées et une organisation méthodologique structurée.

La mise en place d'une architecture client-serveur autoritaire a permis d'aborder concrètement les problématiques de synchronisation, de gestion d'état et de modularité logicielle. L'intégration d'un système de lobby, de rooms indépendantes et d'une boucle de jeu centralisée a structuré le projet autour d'une architecture robuste et évolutive.

L'approche inspirée des méthodes Agile s'est révélée particulièrement adaptée au contexte du projet. Le découpage en sprints courts a permis une progression incrémentale maîtrisée, une validation régulière des choix techniques et une adaptation rapide face aux difficultés rencontrées. La phase de prototypage initiale a réduit les incertitudes, tandis que les itérations intensives de janvier ont permis de transformer cette base en un système stable et jouable.

Au-delà des résultats techniques obtenus, ce projet a permis de développer une compréhension approfondie des enjeux liés au travail collaboratif, à la structuration d'un projet logiciel complexe et à la nécessité d'une communication continue dans un environnement distribué.

L'expérience acquise constitue un socle solide pour aborder des développements futurs, qu'il s'agisse de projets académiques plus avancés ou d'applications professionnelles impliquant des architectures distribuées et des contraintes de robustesse élevées.

Pour finir, ce projet illustre l'importance d'une architecture pensée, d'une organisation claire et d'une grande capacité d'adaptation dans la réussite d'un développement logiciel.